TREE WALKS

A node has two children. One waits while you finish the other.

Recursion: Python remembers the waiting child. Stack: you remember it (push now, pop later).

DFS uses a stack. A queue is not DFS. BFS uses a queue. A stack is not level order.

THE OPTIONS

```
pre   = me, left, right
        recursion
        iterative, 1 stack

in    = left, me, right
        recursion
        iterative, 1 stack
        same recursion with the sides swapped: right, me, left
        morris: no stack, no recursion
        try morris_inorder_try.py

post  = left, right, me
        recursion
        iterative, 1 stack, then reverse
        iterative, 2 stacks          (same walk as the reverse)
        iterative, 1 stack, visit twice
                                     (only if reverse and a second
                                      stack are both banned)

BFS   = row by row
        queue
        kth level is that same row, stopped at row k
        zigzag: every other row is reversed
        try zigzag_level_try.py
```

A BST search, insert, min, max, or remove is not one of these orders. You follow one child. That section is at the end.

Some questions are not "print in this order" either. They are "this node is enough" or "I need the children's answers first".

PREORDER — me, left, right

Recursion. Do the work before either child.

```
def pre(node):
    if node is None:
        return
```

```
    # preorder means me, then left, then right
    print(node.data, end=" ")
    pre(node.left)
    pre(node.right)
```

Iterative, 1 stack. Print as soon as you pop the node. Push right first, then left, so left comes off next. Tutorial Q3.

```
if root is None:
    return
s = Stack()
push(s, root)
#run until all nodes have been added and removed
while not is_empty(s):
    cur = pop(s)
    # want to print now, then do left before right
    print(cur.data, end=" ")
    # stack gives back the last thing pushed, so push right first
    # left sits on top and comes out next
    if cur.right: push(s, cur.right)
    if cur.left:  push(s, cur.left)
```

INORDER — left, me, right

Recursion. Do the work between the two children.

```
def inorder(node):
    if node is None:
        return
    # inorder means left, then middle, then right
    inorder(node.left)
    print(node.data, end=" ")
    inorder(node.right)
```

Iterative, 1 stack. Assignment 2 Q1.

```
if root is None:
    return
s = Stack()
current = root

# inorder means left, then middle, then right
# want to save the node, traverse left, print left, print middle,
# then traverse the right subtree and keep doing so

while current or not is_empty(s):
    #keep moving left and adding the middle nodes until none is hit
```

```
    while current:
        push(s, current)
        current = current.left

    # once we hit none (extreme left) (take out the last node)
    current = pop(s)
    print(current.data, end=" ")

    if current.right:
        #this makes the current not none again for the loop and hence moves down
the extreme left of right subtree
        current = current.right
    else:
        #if no right subtree, next will jump to pop which should be the middle one
        #and then check for its right subtree if not none again and moves to the
parent middle one
        current = None
```

Right, me, left. Same inorder function. Swap the two calls. A BST visited this way goes from large to small. Assignment 2 greater-sum tree: everything already visited is bigger, so a running total is the sum of greater keys. Store it in the node and pass the new total down.

```
def greater(node, total):
    if node is None:
        return total
    # right then me then left, so the bigger keys come first
    total = greater(node.right, total)
    # everything already visited is bigger, add me into that
    total = total + node.data
    node.data = total
    # pass the new total down to the smaller keys
    return greater(node.left, total)
```

Call it as greater(root, 0). The total has to be returned. Passing it as a normal argument does not update the caller.

POSTORDER — left, right, me

Recursion. Do the work after both children.

```
def post(node):
    if node is None:
        return
    # postorder means left, then right, then me
    post(node.left)
    post(node.right)
    print(node.data, end=" ")
```

You cannot print yourself until both children are done. The two iterative versions are the same walk. One uses a second stack to flip the order. The other uses a list and reverse().

Walk recorded: me, right, left After the flip: left, right, me

Iterative, 2 stacks. First stack does not print. It saves every node on the second stack. Emptying the second stack is the flip. Push left then right, so the saved order is me, right, left. Assignment 2 Q5.

```
if root is None:
    return
s1, s2 = Stack(), Stack()
push(s1, root)
while not is_empty(s1):
    cur = pop(s1)
    # can't print yet, both kids have to go first, so save it on stack2
    push(s2, cur)
    # push left then right, saved order becomes me, right, left
    if cur.left:  push(s1, cur.left)
    if cur.right: push(s1, cur.right)
while not is_empty(s2):
    # emptying stack2 flips that to left, right, me
    print(pop(s2).data, end=" ")
```

Preorder is that first loop with two changes: it prints immediately, and it pushes right then left.

Iterative, 1 stack. Same walk. A list replaces the second stack. reverse() does the flip. Lab postOrderIterativeS1, Solution.

```
s = [root]
result = []
while s:
    cur = s.pop()
    # recording now is too early: me, right, left
    result.append(cur.val)
    if cur.left:  s.append(cur.left)
    if cur.right: s.append(cur.right)
# reverse flips it to left, right, me
result.reverse()
return result

recorded order = me, right, left
after reverse  = left, right, me
```

That flip only makes postorder.

Iterative, 1 stack, visit twice. Same left-right-me result. Use this only when you cannot reverse a list and cannot use a second stack. Lab postOrderIterativeS1, Solution02.

On the way left, push the right child first as a reminder, then push yourself. You cannot record yourself yet.

When you pop yourself: reminder still on top -> right side is not done. put yourself back, go right. reminder gone -> both sides are done. Record yourself.

```
if root is None:
    return []
stack = []
current = root
result = []
while current or stack:
    while current:
        if current.right:
            # push right first, reminder that the right side still needs doing
            stack.append(current.right)
        stack.append(current)
        current = current.left
    # once we hit none, pop
    current = stack.pop()
    # reminder still on top means right isn't done, put myself back and go right
    if current.right and stack and stack[-1] is current.right:
        stack.pop()
        stack.append(current)
        current = current.right
    else:
        # reminder is gone, both sides are done, so record me
        result.append(current.val)
        # none so we don't walk left again
        current = None
return result
```

BFS — queue, row by row

DFS finishes one side before the other, so it uses a stack. BFS finishes a whole row before the next row, so it uses a queue. Tutorial Q2.

```
from collections import deque
q = deque([root])
while q:
    # want this row now, so take from the front
    cur = q.popleft()
    print(cur.data, end=" ")
    # kids go at the back, they become the next row
    if cur.left:  q.append(cur.left)
    if cur.right: q.append(cur.right)

stack: append + pop from the end    = DFS
queue: append + pop from the front  = BFS
```

kth level. The nodes k steps down. Same idea as one row of the queue. Recursion counts k down and stops at 0.

```
def kth(node, k):
    if node is None:
        return []
    # k == 0 means we hit the row we want
    if k == 0:
        return [node.data]
    # otherwise go one level down on both sides
    return kth(node.left, k - 1) + kth(node.right, k - 1)
```

THIS NODE IS ENOUGH, OR THE CHILDREN ANSWER FIRST

These are not a fourth order. They are what you do when the question is a job on the nodes, not "print pre / in / post".

This node is enough. Look at it, then visit the children. Order usually does not matter.

```
has a grandchild?   does my child have a child?
print if < m?       look at this item
smallest?           compare this item to the running min
mirror?             swap this node's children, then visit both

stack = [root]
while stack:
    cur = stack.pop()
    # do the job on this node
    # then push the kids so they get visited later
    if cur.left:  stack.append(cur.left)
    if cur.right: stack.append(cur.right)
```

The check is on cur. The push only means "visit this later". The same jobs also fit the recursive preorder shape: work, then both children.

The children's answers have to come back first.

```
height(me) = 1 + max(height(left), height(right))
identical  = both sides must match
sum of non-leaves, subtree sums
```

Recursion: the call returns the child's answer, then you use it. Tutorial Q4 (max depth), lab hasGreatGrandchild, lab identical, Assignment 2 Q3 (sum of non-leaves).

```
def height(node):
    if node is None:
        return -1
    # need both kids' heights before I can answer
```

```
    # me is 1 plus the taller one
    return 1 + max(height(node.left), height(node.right))
```

Empty child returns the empty answer (0 for a sum, -1 or 0 for height, True or False for a check). A sum returns 0, not None.

Postorder is this second kind: me is last because both children go first. That is why postorder has the flips and the visit-twice loop above. Height, great-grandchild, and identical stay as recursion.

BST — follow one child

A BST keeps smaller keys on the left and bigger keys on the right. Search, insert, min, and max never visit both children. They compare and take one side. The lab field is often item. These snippets use data. Same field.

Search.

```
def search(node, value):
    if node is None:
        return False
    if node.data == value:
        return True
    # smaller goes left, bigger goes right
    if value < node.data:
        return search(node.left, value)
    return search(node.right, value)
```

Insert. Walk until the child on that side is empty, then attach the new node. Equal keys are left alone.

```
def insert(node, value):
    # once we hit none, that empty spot is where the new node goes
    if node is None:
        return Node(value)
    if value < node.data:
        node.left = insert(node.left, value)
    elif value > node.data:
        node.right = insert(node.right, value)
    # equal key, leave the tree alone
    return node
```

The iterative insert is the same walk without the call. Keep the parent, because the empty spot has no node to hang the new one on.

```
current = root
parent = None
while current:
    # keep parent, once current becomes none there is no node to hang the new one
on
```

```
    parent = current
    if value < current.data:
        current = current.left
    elif value > current.data:
        current = current.right
    else:
        return root
if value < parent.data:
    parent.left = Node(value)
else:
    parent.right = Node(value)
```

Min is "keep going left". Max is "keep going right".

```
def find_min(node):
    while node.left:
        node = node.left
    return node

def find_max(node):
    while node.right:
        node = node.right
    return node
```

isBST. Every node must sit inside a range inherited from its ancestors. Left child: same low, high becomes me. Right child: low becomes me, same high.

```
def isBST(node, low=float("-inf"), high=float("inf")):
    if node is None:
        return True
    # node has to sit inside the range from its parents
    if not (low < node.data < high):
        return False
    # left side, the high becomes me. right side, the low becomes me
    return (isBST(node.left, low, node.data)
            and isBST(node.right, node.data, high))
```

Remove. Search for the value first, then one of three cases.

```
no left child     -> the right child takes this spot
no right child    -> the left child takes this spot
two children      -> copy the min of the right subtree into me,
                     then remove that min from the right

def remove(node, value):
    if node is None:
        return None
```

```
    if value < node.data:
        node.left = remove(node.left, value)
        return node
    if value > node.data:
        node.right = remove(node.right, value)
        return node
    # no left, the right child takes the spot
    if node.left is None:
        return node.right
    # no right, the left child takes the spot
    if node.right is None:
        return node.left
    # two kids: copy the smallest on the right into me, then delete that smallest
    succ = find_min(node.right)
    node.data = succ.data
    node.right = remove(node.right, succ.data)
    return node
```

The lab remove returns the node and a 0 or -1 as well. The tree change is still these three cases.